Python Grundlagen
Funktionen in Python

Wiederverwendbarer Code

Studienkolleg · Rahn Education

Lernziele

Am Ende dieser Lektion können Sie:

- Funktionen definieren und aufrufen
- Parameter und Rückgabewerte nutzen
- Den Unterschied zwischen `print` und `return` verstehen

Was ist eine Funktion?

Was ist eine Funktion?

Funktion

Funktion = Wiederverwendbarer Code-Baustein

- Organisiert Code in logische Einheiten
- Vermeidet Wiederholungen
- Macht Programme übersichtlicher
- Wie ein Rezept: einmal definieren, oft verwenden

Syntax

Funktion definieren

Syntax: def funktionsname(parameter):

```
1 def funktionsname(parameter1, parameter2):
2     # Code-Block der Funktion
3     ergebnis = parameter1 + parameter2
4     return ergebnis
```

Schlüsselwörter

- `def` – Leitet Funktionsdefinition ein
- **Parameter** – Eingabewerte (optional)
- `return` – Gibt Ergebnis zurück (optional)

Erstes Beispiel

Beispiel

Einfache Begrüßung

```
1 def begruessung():
2     print("Hallo, wie geht es dir heute?")
3
4 begruessung()
```

Beispiel

Einfache Begrüßung

```
1 def begruessung():
2     print("Hallo, wie geht es dir heute?")
3
4 begruessung()
```

```
Hallo, wie geht es dir heute?
```

Parameter

Funktionen mit Parametern

```
1 def personalisierte_begruessung(name):
2     print(f"Hallo {name}, wie geht es dir?")
```

Funktionen mit Parametern

```python
def personalisierte_begruessung(name):
    print(f"Hallo {name}, wie geht es dir?")
```

```python
personalisierte_begruessung("Aaron")
# Hallo Aaron, wie geht es dir?

personalisierte_begruessung("Maria")
# Hallo Maria, wie geht es dir?
```

Mehrere Parameter

```
1 def addiere(a, b):
2     ergebnis = a + b
3     print(ergebnis)
4
5 addiere(4, 5)    # Ausgabe: 9
```

Mehrere Parameter

```python
def addiere(a, b):
    ergebnis = a + b
    print(ergebnis)

addiere(4, 5)    # Ausgabe: 9
```

```python
def volumen_rechteck(l, b, h):
    volumen = l * b * h
    return volumen

v = volumen_rechteck(5, 4, 3)
print(v)    # Ausgabe: 60
```

return vs. print

return vs. print

Mit print

```
1 def addiere_print(a, b):
2     ergebnis = a + b
3     print(ergebnis)
4
5 x = addiere_print(3, 4)   # Ausgabe: 7
6 print(x)                  # Ausgabe: None
```

Achtung

`print` gibt **nichts zurück** – die Variable `x` enthält `None`.

return vs. print

Mit return

```python
def addiere_return(a, b):
    ergebnis = a + b
    return ergebnis

y = addiere_return(3, 4)   # Keine Ausgabe
print(y)                   # Ausgabe: 7
print(y * 2)               # Ausgabe: 14
```

`return` gibt den Wert zurück – `y` enthält `7` und kann weiterverwendet werden.

Häufiger Fehler

print statt return

```
x = addiere_print(3, 4)
print(x * 2)
# TypeError: unsupported operand type(s)
# for *: 'NoneType' and 'int'
```

Benannte Parameter und Standardwerte

Benannte Parameter

```
1 def beschreibe_person(name, alter, stadt):
2     print(f"{name} ist {alter} Jahre alt und
        lebt in {stadt}.")
```

Benannte Parameter

```python
def beschreibe_person(name, alter, stadt):
    print(f"{name} ist {alter} Jahre alt und lebt in {stadt}.")
```

```python
# Positionsbasiert
beschreibe_person("Lisa", 25, "Berlin")

# Mit benannten Parametern (Reihenfolge egal!)
beschreibe_person(stadt="Muenchen", name="Tom", alter=30)
```

Standardparameter

Default-Werte

```
1 def addition(a, b=3):
2     return a + b
```

Standardparameter

Default-Werte

```
1 def addition(a, b=3):
2     return a + b
```

```
print(addition(5))        # Ausgabe: 8  (5 + 3)
print(addition(5, 10))    # Ausgabe: 15 (5 + 10)
```

Wenn kein Wert für `b` angegeben wird, wird der Standardwert `3` verwendet.

Funktionen + Kontrollstrukturen

Funktionen + Kontrollstrukturen

```
1 def pruefe_zahl(zahl):
2     if zahl > 0:
3         return "Die Zahl ist positiv."
4     elif zahl < 0:
5         return "Die Zahl ist negativ."
6     else:
7         return "Die Zahl ist null."
```

Funktionen + Kontrollstrukturen

```python
def pruefe_zahl(zahl):
    if zahl > 0:
        return "Die Zahl ist positiv."
    elif zahl < 0:
        return "Die Zahl ist negativ."
    else:
        return "Die Zahl ist null."
```

```python
for i in range(-2, 3):
    print(f"Fuer {i}: {pruefe_zahl(i)}")
```

Funktionen rufen Funktionen auf

Funktionen rufen Funktionen auf

```python
def eingabe_verarbeiten():
    name = input("Wie heisst du? ")
    return name

def begruessung_anzeigen(person):
    print(f"Herzlich willkommen, {person}!")

def hauptprogramm():
    benutzer = eingabe_verarbeiten()
    begruessung_anzeigen(benutzer)

hauptprogramm()
```

Praktisches Beispiel

Beispiel

Einfacher Rechner

```
1 def berechne(a, b, operation="addiere"):
2     if operation == "addiere":
3         return a + b
4     elif operation == "subtrahiere":
5         return a - b
6     elif operation == "multipliziere":
7         return a * b
8     else:
9         return None
```

Beispiel

Einfacher Rechner

```
1 def berechne(a, b, operation="addiere"):
2     if operation == "addiere":
3         return a + b
4     elif operation == "subtrahiere":
5         return a - b
6     elif operation == "multipliziere":
7         return a * b
8     else:
9         return None
```

Übungen

Mini-Übung 1

Funktion: verdopple

Schreibe eine Funktion `verdopple(zahl)`, die:

1. Eine Zahl als Parameter nimmt
2. Die Zahl mit 2 multipliziert
3. Das Ergebnis **zurückgibt** (nicht ausgibt!)

Teste die Funktion mit `verdopple(5)`

Lösung: Mini-Übung 1

```python
def verdopple(zahl):
    return zahl * 2

ergebnis = verdopple(5)
print(ergebnis)  # 10
```

Mini-Übung 2

Funktion: ist_gerade

Schreibe eine Funktion `ist_gerade(zahl)`, die:

1. Prüft, ob die Zahl gerade ist
2. `True` zurückgibt, wenn gerade
3. `False` zurückgibt, wenn ungerade

Tipp: Nutze den Modulo-Operator %

Lösung: Mini-Übung 2

```python
def ist_gerade(zahl):
    if zahl % 2 == 0:
        return True
    else:
        return False

# Oder kuerzer:
def ist_gerade(zahl):
    return zahl % 2 == 0

print(ist_gerade(4))    # True
print(ist_gerade(7))    # False
```

Große Übung

Funktionen kombinieren

Schreibe ein kleines Programm mit drei Funktionen:

1. `hole_zahlen()`: Fragt nach zwei Zahlen mit `input()`
2. `multipliziere(a, b)`: Multipliziert zwei Zahlen
3. `zeige_ergebnis(zahl)`: Gibt das Ergebnis aus

Rufe alle drei Funktionen nacheinander auf!

Lösung – Teil 1

Funktionen definieren

```
def hole_zahlen():
    a = int(input("Erste Zahl: "))
    b = int(input("Zweite Zahl: "))
    return a, b

def multipliziere(a, b):
    return a * b

def zeige_ergebnis(zahl):
    print(f"Das Ergebnis ist: {zahl}")
```

Lösung – Teil 2

Hauptprogramm

```
a, b = hole_zahlen()
ergebnis = multipliziere(a, b)
zeige_ergebnis(ergebnis)
```

Tipp

Zahlen eingeben → Multiplizieren → Ergebnis anzeigen

Best Practices

Best Practices

Gute Funktionen sind:

- **Beschreibend benannt:** `berechne_durchschnitt()` statt `func1()`
- **Fokussiert:** Eine Funktion = eine Aufgabe
- **Wiederverwendbar:** Allgemein formuliert
- **Dokumentiert:** Kommentare erklären den Zweck

Zusammenfassung

Zusammenfassung

- Funktionen definieren mit `def`
- Parameter für flexible Funktionen
- `return` gibt Werte zurück (wichtig!)
- Standardwerte für Parameter möglich
- Funktionen können andere Funktionen aufrufen

Hausaufgaben

Hausaufgaben

Aufgaben

1. Schreibe eine Funktion `berechne_summe(liste)`, die alle Zahlen in einer Liste addiert
2. Erstelle eine Funktion `finde_maximum(a, b, c)`, die die größte von drei Zahlen zurückgibt
3. Schreibe eine Funktion `ist_palindrom(wort)`, die prüft, ob ein Wort vorwärts und rückwärts gleich ist

Fragen?